

Snowflake

* It is a cloud based platform.

* There are 3 layers

Storage layer

Compute layer

Cloud services layer.

* It is a database house.

Q) How to create a file format?

CREATE FILE FORMAT Custom-CSV-FORMAT (File-Format-name)

TYPE = CSV

Field-Delimiter = ','

SKIP-Header = 1

Date-Format = 'DD-MM-YYYY';

A If data is not coming proper format at that time we will be file format.

Q) How to create temp table?

* Table will be dropped here not truncated after the session.

CREATE TEMP TABLE table-name (col1 datatype, ...).

for transient: * It is like permanent only one thing is different it don't have file-format.

CREATE TRANSIENT TABLE table-name (col1 datatype, ...).

for external:

CREATE EXTERNAL TABLE table-name (

location 'path';

* Tables are present in some other locations like AWS S3,

Azure blob storage etc.

For ice berg table:-

Create ice berg table table-name ();

For hybrid:-

Create hybrid table table-name (col1 datatype, ...).

* It must have primary key.

Warehouse Creating by using SQL:-

* First we have to be in sysadmin role.

OR role SYSADMIN;

Create or replace warehouse warehouse-name
with

Warehouse-size = 'small'

MIN-CLUSTER-COUNT = 1

MAX-CLUSTER-COUNT = 3;

Drop warehouse warehouse-name; to drop

We can able to edit the warehouse by using Alter command
Ex:-

1. Alter warehouse warehouse-name suspend;

2. Alter warehouse warehouse-name set Warehouse-size = 'small'.

Copy Into Command:-

* By using we can ~~load~~ load the data into tables

Copy into table-name

from 'Path'

file-format = (type = CSV

field-delimiter = ','

Skip-header = 1);

Showflase editions:-

1. Standard (very basic) 2 \$/Credit
2. Enterprise (some security & all features) 3 \$/Credit
3. Business critical (Private) 4 \$/Credit
4. Virtual private (more private dedicated servers, Showflase accounts).

Pricing:-

ED Compute

- Warehouse (Active) $\rightarrow$ based query processing.
- Cloud services - Showflase take care of this
- Servers

ED Storage

- * 40 \$ / TB / month.
- * monthly storage fees
- * It calculate daily basis.

* Prices will differ based on region & edition like many factors.

Roles:-

User $\leftarrow$ Role $\leftarrow$ Privilege

- * One user may have multiple roles.
- * Privileges are assigned to roles, user are assigned to roles.

Loading Data

* There are 2 types

1. Bulk Loading.
2. Continuous Loading.

* While loading data into staging if we are getting data-related errors or any other error in last line we have to mention
on-error = 'continue';

Ex for load:

Copy into target-table from (select * from table-name)
file-format = (type = csv field-delimiter = ',' SKP-header 21)
files = ('OrderDetails-err01.csv')

on-error = 'Continue';

* In files we can pass multiple files

on-error Ex:-

1. Continue → It skips those rows & continue all records.

2. ABORT-STATEMENT → It terminate if any error occurs

3. SKIP-FILE → It skips that particular file.

4. SKIP-FILE-3 → 3 indicates how many rows have to be fail then only that file will be terminated. It is like error limit.

5. SKIP-FILE-5% → Based on percentage it is going to skip file.

File format:

Create or replace file format file-format-name;

Size limit:

Copy into table-name from external-storage

file-format = ()

files = ()

Size-Limit = num

* num ~~must~~ in terms of bytes.

* If limit reaches in one particular file, it completes full file load then it stops loading.

Ex:- limit is 2000 bytes but our first file size is 5000 bytes then it loads total 5000 bytes of data stops further only it won't go to next file.

Default = False

Return failed only:-

Copy into table.name

from external-stage

file-format = ()

files = ()

Return-failed-only = True / False

True → It truncates the target column

False → It gives error

Default = false

* For true based on length limit it will store after that length it removes.

Force:-

Copy into table.name

from external-table

file-format = ()

files = ()

Force = True / False

Default = False

* Specific to load all files, regardless of whether they've been loaded previously & have not changed since they were loaded.

Load unstructured data:-

1. Create stage
2. Load raw data
3. Analyse & parse
4. Flatten & Load

* Select Raw-File: city from table-name;

To fetch city details from 1 column by using ":"

* Select \$1: city from table-name;

Same like above one.

* Select \$1: First-name :: String as first-name as Table-name;

To change data type by using "::"

Ex:

select

Raw-File :id :: int as id,

Raw-File :first-name :: String as first-name

Raw-File :last-name :: String as last-name

Raw-File :gender :: String as gender.

from Table-name;

* To handle nested data by using "." we can achieve it
select Raw-File :job.Salary :: int as salary from table-name

* To find array-size:-

select array-size (Raw-File :spoken-language) from TD.

Snowpipe:-

- * Enable loading once a file appears in a bucket.
- * Snowpipe uses serverless features intd of warehouse.
- * If data needs to be available immediately for analysis.

Setting up snowpipe: AWS:-

1. Create stage → To have the connection.
2. Test copy Command → To make sure it works
3. Create pipe → Create pipe as object with copy Command
4. S3 notification → To trigger snowpipe.

Pipe creation:-

create or replace pipe manage-DB. pipe employee-pipe

~~auto-ingest~~ = auto-ingest = True

AS

copy into owl-first-DB. Public-employees

from @manage-DB. external-stages.csv-folder;

desc pipe pipe-name; here we can get channel Id.

alter pipe pipe-name refresh; To refresh pipe & get last updated
Handle error:- file names.

select SYSTEM\$PIPE_STATUS('pipe-name'); (Validate pipe is
(To check the status of pipe) actually working).

select * from table (validate-pipe-load

pipe-name => 'pipe-name'); snowpipe error message.

Select * from table (Information - schema copy - History (table - name => 'pipe - name')) - copy command history from table to see error message

Some pipe commands:-

1. Desc pipe pipe-name;

2. Show pipes;

3. Show pipes like 'x.employee.y';

4. Show pipes in database database-name;

5. Show pipes in schema database.schema-name;

6. Show pipes like 'x.employee.y' in database database-name;

7. Alter pipe pipe-name set pipe - Execution - paused = True;

* To pause the pipe if we are changing anything related to pipe

* To replace 'True' to 'False' to activate pipe.

Setting up snowpipe for Azure:-

1. Stage Integration -> * Connection details to container
* Grant permission.

2. Create Stage -> Location to Container

3. Queue + Notification -> To trigger snow pipe

4. Notification integration -> * Notification can be received by snowflake
* Grant permissions.

5. Create pipe -> Create pipe as object with copy command.

Time Travel:-

* Query deleted or updated data.

* Restore data, schema & database that have been dropped.

* Create clones of tables, schemas & database from previous state.

Query historic data within retention period:-

1. Timestamp $\rightarrow$ select * from table AT (timestamp $\Rightarrow$ timestamp)

2. Offset $\rightarrow$ select * from table AT (offset $\Rightarrow$ -10*60)

3. Query $\rightarrow$ select * from table before (statement $\Rightarrow$ query-ID)

* Default retention period is 1 day.

* max is 90 days

Alter table table-name set DATA-RETENTION-TIME-IN-DAYS=2;

* Above is syntax for changing retention period.

Fail Safe:-

* Protection of historical data in case of disaster.

* Non-Configurable 7-days period for permanent tables.

* period starts immediately after time travel period ends.

* No user interaction & recoverable only by snowflake support team

* Storage Cost

Table types:-

1. Permanent table $\rightarrow$ Time travel, Fail safe, no billing

2. Transient table $\rightarrow$ Time travel, No fail safe, less billing compare to permanent

Syntax:- Create transient table

3. Temporary table $\rightarrow$ Time travel, No fail safe

* It only present in that session only this the only difference with transient

Syntax:- Create temporary table

* We can able to create database or schema level of transient

Ex: create transient database database-name;

create transient schema schema-name;

* Inside if we can tables it automatically consider that table type.

Continuation of ^{DATE} snowflake

Zero-copy cloning:-

* It clones the schema, data, database, ---

Syntax:- create table table-name clone table-source.

* It supports for

- | | |
|-------------|----------------|
| 1. Database | 5. File format |
| 2. schema | 6. Sequence |
| 3. Table | 7. stage |
| 4. Stream | 8. task |
| | 9. pipe |

* Pipe & stage has some limitations.

* Pipe can only clone if they are only external stages

* Name internal stages not be cloned.

* Privileges will be always only be inherited to child objects never to source object itself. For source object administrator have to give privileges again.

* Cloned tables are independent.

* We can't ~~create~~ be cloned a permanent table with temporary table

?

* But we can clone transient table with temporary table

* for temporary to temporary also allowed.

* For storage we don't need to pay extra amount unit we perform update or insert or delete in the cloned table.

Data sharing:-

It is used to share data the client will use his computer

Setting up share:-

1. Create share:-

Create share my-share;

2. Grant privileges to share:-

Grant usage on Database my-db to share my-share;

Grant usage on schema my-schema.mydb to share my-share;

Grant select on table my-table to share my-share;

3. Add Consumer account:-

Alter share my-share add Account b167131;

4. Import share:-

Create database my-db from share my-share;

Some commands:-

* Show grants to share my-share; → validate grants.

* Show shares;

* Desc share share-details;

By using interface:-

Data products → Private sharing → share

* We can able to share only one database data only.

* To overcome it we will use several views to share multiple databases data to client.

- * If we are sharing data with non-showflake user means we have to pay for storage & compute.
- * In 2 step we can include like this also for all tables
Grant select on all tables in schema db.schema to share my-share;
Grant select on all tables in database db to share my-share;

Data Sampling

- * Getting a sample data from huge data to perform tasks.

Ex:- select * from table table-name sample row (1) seed (27);

Scheduling tasks

Syntax:

create or replace task task-name

warehouse = warehouse-name

schedule = '1 minute'

as

Insert into customers (create_date) value (sysdate);

* Show tasks;

* Default task is suspended we have to start the task.

* Alter task task-name Resume;

* Alter task task-name Suspend;

by using cron :-
schedule = 'Using cron'
min(0-59) Hour(0-23) Day of month(1-31) month(1-12) day of week(0-6, sun-SAT) Time zone (UTC)

Ex:-
schedule = 'Using cron 0 20-23 ~~1,15~~ 1-12 mon-Fri UTC'

Tree of tasks:-

* It is like DAG in Airflow

* If we are doing any kind of changes like adding child tasks or removing child tasks we have to suspend root task.

Syntax to create for child task:-

Create or replace task task-name

Warehouse = warehouse-name

After root-task-name

AS

Insert into table table-name : (Any SQL statement)

* If it is a big DAG we have to enable each & every tasks it some what complicated so we will use this command.

* select system \$task-Dependents - Enable ('root-task-name');

Streams

- * Object that records changes made to a table
- * This process is called change data capture (CDC)
- * It is going to have extra 3 columns by default

METADATA\$ACTION, METADATA\$UPDATE, METADATA\$ROW-ID.

Syntax: Create & replace Stream stream-name on table table-name;

- * select * from stream-name;
- * After Consuming data from the streams the data will be removed.

* Show streams;

* Desc stream stream-name;

* For update:-

merge into target-table +

using Source-table S

on t.id = S.id

When matched & S.METADATA\$action = 'Insert' AND

S.METADATA\$update = 'True'

then update.

Set T.col = S.col

T.col = S.col;

* Source is Stream table.

For delete:

merge into target t

using source s

on t.id = s.id

when matched & s.METADATA\$action = 'Delete' AND

s.METADATA\$isupdate = 'False'

then delete;

* source is stream table.

Using all 3 in same time:-

merge into target-table t

using source-table s

on t.id = s.id

when matched & s.METADATA\$action = 'Delete' & s.METADATA\$isupdate

then delete -- Delete

= 'False'

When matched & s.METADATA\$action = 'Insert' &

s.METADATA\$isupdate = 'True'

then update -- update

set t.col = s.col

t.col = s.col

When not matched & s.METADATA\$action = 'Insert'

then insert -- Insert.

(col, col, col, ...) values (val1, val2, val3, ...);

Combining task & stream:-

Create & replace task all-data-changes

Warehouse = "warehouse-name"

Schedule = '1 minute'

When system & stream has data ('stream-name')

AS It return true & Page based on that task will execute.

merge statement [Same as previous page statement]

Types of streams:-

1. Standard → supports Insert, update, Delete
2. Append-only → ~~supports only~~ ^{Insert} ~~update~~ ^{* Better for performance} only for insert.

Syntax:- Create & replace stream stream-name
on table table-name APPEND-only = True;

Materialized views

- * By using this we can get upto date data.
- * It uses less compute power to execute same query.
- * It store query & data
- * It changes more compare to view.

Syntax:- Create & replace materialized views AS (select command)

Limitations:-

- * Joins not allowed, UDF, Having, Order by, LIMIT not supported
- * Limited aggregation

Data masking

* Hiding data to some particular roles

* Column level hiding data.

Syntax: 1. Set up masking policy

Create or replace masking policy ~~name~~ masking-name
as (val varchar) returns varchar ->

case

when current-role() in ('ANALYST_Full', 'ROLE2', ...) then
val

else '##-####-##'

end;

* Here we are making allow to see those roles mentioned
in query remaining roles are not able to see After
applying these policies to particular column.

2. Apply policy on a specific column.

Alter table IF EXISTS table-name modify column column-name
set masking policy masking-policy-name;

* If we want to drop or ^{replace} the policy we have to do unset

* Desc masking policy policy-name;

* Show masking policies;

* Select * from table Cinformation-schema.policy-reference
(policy-name => 'masking-policy-name');

* It shows columns with applied policy

- * Remove policy before replacing/dropping.
- Alter table if exists table-name modify column column-name
use + masking policy;
- * To get first 2 letters we have to change else like.
else concat (left (val, 2), '****')
- * To update existing masking policy.
alter masking policy masking-policy-name set body ->
case
when curren_role() in ('role1', 'role2', ...) then val
else concat(left (val, 3), '*****')
end;
- * Each column can associate at 1 policy only.

Access Control

- * Who can access & perform operations on objects in snowflake
- * Two aspects of access control combined.
- 1. Discretionary Access Control (DAC) :-
 - * Each object has an owner who can grant access to that object
- 2. Role based Access Control (RBAC) :-
 - * Access privileges are assigned to roles which are in turn assigned to users.

Ex: Grant role > to <user>

* Assigning a role to user.

Grant <privilege> on <object> To <role>

* Giving privileges to object to particular role.

* There are 5 system define roles.

1. Account Admin: Account admin tab, Billing & usage, Reader Account.

* SysAdmin & security admin

* top-level role in the system.

* Should be granted only to a limited no. of users

* MFA & create other users.

This things
we can
perform.

2. Security Admin: user, role, policy, sessions & resource monitors

* Useradmin role is granted to securityadmin

* Can manage users & roles

* Can manage any object grant globally.

* Grant & revoke privileges to roles.

3. SysAdmin:

* Create warehouses & database (& more objects)

* Recommended that all custom roles are assigned.

4. usradmin:-

- * Dedicated to user & role management only.
- * Can create users & roles.

5. Public:-

- * Automatically granted to every user.
- * Can create own objects like every other role (available to every other user/role)

How to create user:-

create user user-name password = '123'

default-role = ACCOUNTADMIN

must-change-password = true;

* Grant role Accountadmin to user user-name;

* For default-role we can use accountadmin, securityadmin & sysadmin.

How to create role:-

create role role-name;

create hierarchy:-

~~create~~ grant role role₂ to role role₁;

A role₁
↓
role₂ like this it will assume.

~~create~~ as per but practice assign role to sysadmin

grant role₁ to role sysadmin;

sysadmin } like this it
role₁ } will create.
role₂

Best practice

Warehouse :- for warehouse cost

1. Enable auto suspend → Based on own work style or work load.
2. Enable auto resume
3. Set appropriate timeouts.
4. Choose size based on workload

Table design :- for storage cost

1. Appropriate table type

Raw (δ) staging ^{table} → ^{use} Transient table

Silver (δ) development tables → use transient table

Gold (δ) productive table → use permanent table

2. Appropriate data type.

3. Set cluster keys only if necessary.

Retention period :-

1. Staging database → 0 days (Transient)
2. Production → 4 - 7 days (1 day min)
3. Large high-churn tables → 0 days.

Snow SQL

- * It is like a Command Line Interface (CLI)
- * Used to automate the tasks, CI/CD pipeline like that.
- * We have to install it from browser then configure in CMD by using some commands.
- * `snowsql -a <account-name> -u <user-name>`
- * Above command gives all ^{pathways} ~~paths~~ for particular user
- * `snowsql -a <account-name> -u <user-name> -w <warehouse-name> -d <data-base-name>`
- * To send local files to stage using CLI:-
`put file : //C: user: path @stage-name;`
 must

Some CLI Commands:-

1. Current status:-

1. select current-database();
2. select current-user();
3. select current-schema();
4. select current-warehouse();

To copy data to local stage by using CLI:-

`get @stage-name /table-name file:// path`
 must.

To remove files in stage:-

`remove @stage-name;`

Stages

* There are mainly 3 types of stages.

1. User stage:- (@v)

* It is created automatically

we can access it by using '@v'

* This stage for user level means * This stage associated to that user.

2. Table stage:- (@v, table_name)

* This ^{stage} ~~table~~ for table level means * This stage associated to that table

* We can access it by using '@v, table_name'

* It is created automatically.

3. Named user:-

* It is divided into 2 types

(i) Name internal stage:- (@stage_name)

* We have to create it manually.

create stage stage_name;

(ii) Named external stage :- (@stage-name)

- * we have to create it manually
- * An external stage points to files stored in your cloud storage.

- * snowflake reads files doesn't store them.

create stage stage-name

URL = 'URL'

STORAGE_INTEGRATION = stage-integration-name;